

1. ***What is Named Entity Recognition (NER)?***

NER is the classification of tokenized keywords that are categorized into specified groups such as geographical regions (GPE), personal names (PERSON), financial keywords (MONEY), or temporal units (DATE). Such a technique is useful in predicting text that should be succeeding the keywords as well as developing a context for the keyword in use. This is implemented by evaluating the surrounding keyword types and their relationships to each other, its repetition, and its significance matrix.

2. ***Explain how you implemented the entity counting system***

The Python package Pandas has a mathematical subfunction called `values_count()` that returns a Series of counts for unique values in a column, sorted by frequency. I used this to store the count values in a variable and display them on the system console

3. ***How does your bar graph represent the data? What insights can be gathered from the visualization? How could the visualization be improved?***

For this exercise, I elected to use a fictional news document of approximately 4,000 words, as the given sample text was too limited to realistically evaluate the system's behavior. Initially, the bar graph contained several entities with miniscule count values below 10, which rendered a convoluted and confusing presentation of small bar graphics too small to be visually estimated. This prompted me to augment the code so that entities with a count below 15 were grouped together under the label "Other" to

represent all possible insignificant values. This aesthetically improved the bar graph presentation, and it displayed the most significant entity groups ORG, GPE, LOC, DATE, and MONEY. For an improved visual presentation, I additionally included a pie chart to demonstrate the significance in the context of frequency of each word.

4. *Discuss your implementation of the multiword filter.*

For this task, I implemented a chain of subfunctions from the Pandas data frame object:

```
mask = frame["Text"].astype(str).str.split().str.len() == 1
```

In sequence, the algorithm first converts the contents of the data frame object from static values into a dynamic string type. Each string is then split by each whitespace using the subfunction `str.split()`. The last subfunction `str.len()` evaluates the length of each string value and uses the concluding conditional statement `==1` to add only the values containing one string value to the assigned variable `mask`. I finalized it by using the `copy()` subfunction to copy the string values saved to the `mask` variable back into the original data frame, which overwrote the original contents and replaced them with entities containing only a single word.

I noted that when the multiword filter function was applied, the accuracy metrics increased significantly with the F1 score by nearly 15%. This is particularly a result stemming from the elimination of a vast number of multi-worded entities that were falsely identified as positive. However, this also eliminated several other entities that were correctly classified. Fine tuning NER for higher precision and accuracy will require advanced techniques to properly classify entities without disposing of those

that were already correctly identified.

5. *Evaluate the performance of the NER system*

The initial accuracy testing of the NER system was egregiously low with an F1 score of nearly zero and nearly everything in the text sample being misclassified, which I determined was due to the training gazetteer being far too small. To increase the accuracy, I ultimately ended up generating a gazetteer of over 10,000 sample comparison entries to raise the precision rating to approximately 59% and the F1 score to approximate 65%. As mentioned earlier, the elimination of multi-worded entities improved accuracy but also sacrificed a great number of correctly identified entities. The NER system alone does not appear to be satisfactorily accurate and calls for further augmentation to raise its precision scores.

I also observed that the NER system is not ideal for evaluating small text samples—the distribution was better portrayed using larger amounts of entities. With the graph presentations, I was able to identify several significant entities such as ORG, GPE, LOC, DATE, and MONEY while simultaneously noting several insignificant amounts of entities that appeared in the graph. This created noise in the distribution that needed to be cleansed. It caused me to doubt whether these entities were in fact necessary to even name to begin with. My suggestion to improve this system would be to group sporadic-appearing entities under a classification labeled as “Other” or eliminating them altogether.